

Data Progression in Smart Healthcare Systems

Foundational Patient Data 🏥

- Represents raw facts like temperature readings and diagnostic details.
- Acts as the basic building block for all healthcare insights.
- Serves as the input for the smart system's processing engine.
- Identification of Data Objects
 - Individual entities such as a single patient's medical history.
 - Real-time sensor outputs like heart rate or blood pressure.
 - Grouping related values into a single identifiable unit.

Data Structure Organization 📁

- Defines the physical arrangement of data objects in memory.
- Uses pointers to establish relationships between different data points.
- Facilitates efficient storage and retrieval of patient records.
- Structural Relationships
 - Linking patient details to specific diagnostic results.
 - Organizing history chronologically for medical review.
 - Memory Management
 - Utilizing pointers to optimize memory usage in resource-constrained devices.

Abstract Data Type (ADT) 🧠

- Provides a high-level model of 'what' operations can be performed.
- Encapsulates data and operations while hiding implementation details.
- Ensures the healthcare system logic remains independent of coding specifics.
- Logical Operations
 - Defining methods like 'Update Vitals' or 'Query History'.
 - Establishing rules for data access and security.
 - Operational Abstraction
 - Separating the logic of data manipulation from the underlying hardware.

Implementation in C Language 💻

- Translates the ADT into functional code using specific syntax.
- Employs the 'struct' keyword to define complex patient entities.
- Maps the logical model to physical memory addresses.
- Concrete Coding Tools
 - Implementation of linked lists or arrays for record management.
 - Using functions to perform the operations defined by the ADT.
- Systematic Execution
 - Compiling code into machine-executable instructions for the system.

System Performance Impact 🚀

- Proper data organization leads to faster diagnostic processing.
- Hierarchical structures reduce the time complexity of data searches.
- Modular implementation allows for easier system updates and scaling.
- Efficiency and Scalability
 - Optimized resource usage for large-scale patient databases.
 - Reduced latency in real-time health monitoring tasks.
- Performance Justification
 - Linking data design directly to improved medical response times.